SR UNIVERSITY, WARANGAL

DEPARTMENT OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE

PROJECT TITLE

RAILMITRA

A Full-Stack Railway Passenger Assistance and Service Management System

A course project report submitted in partial fulfillment of the requirement of

AI ASSISTANT CODING

By

M. Vikas (2303A51155)

Under the guidance of

Salauddin Sir

SR University, Warangal

APRIL – 2026

ABSTRACT

RailMitra is a full-stack web-based railway assistance platform designed to connect passengers with on-ground assistants at stations through a centralized digital workflow. The primary objective of this project is to improve passenger convenience, accessibility, and support efficiency by automating booking, assistant allocation, communication, and administrative monitoring.

The system addresses practical problems faced by elderly passengers, differently abled travelers, and users requiring station support for navigation, luggage handling, and time-sensitive coordination. RailMitra provides role-based modules for passengers, assistants, and administrators, enabling transparent service request handling and real-time communication.

The application is built using Node.js, Express.js, MongoDB, and Socket.IO. It includes authentication, booking workflows, service task management, feedback handling, and support ticket flows.

The project demonstrates that integrating full-stack architecture with real-time communication significantly improves service coordination, response visibility, and operational control.

CONTENTS

Chapter No	Title	Page No
1	INTRODUCTION	01
1.1	About the Project	01
1.2	Problem Statement	02
1.3	Objectives	03
1.4	Scope of the Project	04
2	LITERATURE REVIEW	05
2.1	Overview	05
2.2	Related Studies	06
2.3	Tools & Technologies	07
3	METHODOLOGY	08
3.1	Requirement Analysis	08
3.2	System Architecture	09
3.3	Database Design	10
3.4	Module Design	11
3.5	Implementation Strategy	12
4	RESULTS AND DISCUSSION	13
4.1	Functional Results	13
4.2	Workflow Outcomes	14
4.3	Performance Discussion	15
4.4	Limitations	16
5	CONCLUSION AND FUTURE WORK	17
5.1	Summary	17
5.2	Conclusion	18
5.3	Future Scope	19

CHAPTER 1: INTRODUCTION

1.1 About the Project

RailMitra is a digital railway assistance system developed to simplify how passengers request and receive station-based support services. It provides a structured workflow where passengers create service bookings, assistants manage tasks, and administrators monitor operations.

The platform caters to a wide range of passengers, particularly elderly travelers, differently abled individuals, and those unfamiliar with railway station layouts. By digitizing service requests and communication, RailMitra removes the dependency on informal systems and manual coordination.

The system is accessible via a web browser and offers three distinct portals: one for passengers to book services, one for assistants to manage and complete tasks, and one for administrators to oversee the entire workflow.

1.2 Problem Statement

Existing railway support systems suffer from several critical challenges that reduce passenger satisfaction and operational efficiency:

- Delayed response to passenger requests due to lack of a centralized system
- Absence of real-time service tracking, leaving passengers uninformed about assistance status
- Communication gaps between passengers, assistants, and station management
- Poor administrative monitoring and lack of performance analytics
- No structured mechanism for feedback collection or issue resolution

These problems collectively result in poor passenger experiences, especially for vulnerable groups who most rely on station assistance services.

1.3 Objectives

The key objectives of the RailMitra project are:

- Provide a user-friendly digital booking platform for railway station assistance
- Automate the allocation of tasks to available and verified assistants
- Enable real-time communication between passengers and assistants via Socket.IO
- Provide administrators with a comprehensive monitoring dashboard
- Implement role-based authentication to ensure data security
- Design a scalable, modular system architecture that can be extended to multiple stations

1.4 Scope of the Project

The scope of RailMitra covers the following functional areas:

- Passenger booking system with multi-step service request workflow
- Assistant task management portal with real-time updates
- Admin panel for monitoring bookings, feedback, audit logs, and user roles
- Real-time support communication using Socket.IO
- Feedback and rating collection for completed services

CHAPTER 2: LITERATURE REVIEW

2.1 Overview

Modern transportation systems increasingly rely on digital workflows to improve efficiency and passenger satisfaction. The integration of real-time data, role-based interfaces, and automated task allocation has proven effective in logistics, healthcare, and public service sectors.

Railway systems worldwide are adopting digital platforms to manage passenger services, and several studies highlight the benefits of centralized service management systems in high-footfall environments such as railway stations.

2.2 Related Studies

- Role-based systems improve accountability by assigning clear responsibilities to each stakeholder in the service chain, reducing ambiguity and errors.
- Real-time communication frameworks such as WebSockets have been shown to significantly improve service speed and transparency compared to polling-based approaches.
- Centralized data management using NoSQL databases like MongoDB provides the flexibility needed for heterogeneous service data in dynamic environments.
- Studies on passenger assistance in public transit highlight that targeted digital tools increase usage rates among elderly and differently abled travelers.

2.3 Tools and Technologies

The following tools and technologies were selected based on their suitability for full-stack real-time web application development:

- Node.js – Server-side JavaScript runtime for handling concurrent requests efficiently
- Express.js – Lightweight web framework for REST API development
- MongoDB – NoSQL document database for flexible data modeling
- Socket.IO – Library for real-time bidirectional event-based communication
- HTML, CSS, JavaScript – Frontend technologies for responsive user interfaces

CHAPTER 3: METHODOLOGY

3.1 Requirement Analysis

Functional Requirements:

- User registration, login, and role-based authentication (Passenger, Assistant, Admin)
- Multi-step service booking workflow with payment and confirmation stages
- Task assignment and management for assistants
- Admin dashboard with booking management, feedback review, and audit logs
- Real-time chat and support ticket system
- Feedback and rating collection post service completion

Non-Functional Requirements:

- Fast API response times with efficient database queries
- Secure authentication with JWT tokens and encrypted credentials
- Scalable design to support additional stations and users
- Responsive UI for both desktop and mobile access

3.2 System Architecture

RailMitra follows a layered architecture comprising five major layers:

- Frontend Layer – HTML/CSS/JavaScript-based responsive UI with three role-specific portals
- API Layer – RESTful APIs built with Express.js for all business operations
- Business Logic Layer – Service allocation, booking management, and notification handling
- Database Layer – MongoDB collections for Users, Bookings, Assistants, Feedback, and Tickets
- Real-Time Communication Layer – Socket.IO for live status updates and chat

3.3 Database Design

The following entities form the core data model of RailMitra:

- User – Stores passenger credentials, contact details, and role information
- Assistant – Stores verified assistant profiles, availability, ratings, and earnings
- Booking – Records service requests including train details, service type, status, and payment
- ServiceTask – Tracks individual tasks assigned to assistants for each booking
- Feedback – Captures ratings and comments submitted by passengers post-service
- SupportTicket – Manages passenger complaints and resolution workflows

3.4 Module-Wise Design

The system is organized into four primary modules:

- Passenger Module – Service browsing, booking creation, status tracking, and feedback submission
- Booking Sub-Module – Multi-step form with Booking Details, Payment, and Confirmation stages
- Assistant Module – Task acceptance, status updates, and performance tracking

- Admin Module – Dashboard analytics, booking management, assistant verification, and audit logs
- Support Module – Integrated chat system and support ticket management

3.5 Implementation Strategy

The project was implemented in the following phases:

- Phase 1: Back-end setup with Node.js and Express.js, including database connection and schema design
- Phase 2: Authentication system with JWT-based role management
- Phase 3: Booking and task assignment system development
- Phase 4: Real-time communication integration using Socket.IO
- Phase 5: Admin dashboard and reporting features
- Phase 6: Testing, bug fixing, and final deployment

CHAPTER 4: RESULTS AND DISCUSSION

4.1 Functional Results

All core features of RailMitra were implemented and tested successfully:

- Booking system successfully creates, manages, and tracks service requests
- Task assignment workflow correctly allocates bookings to available assistants
- Real-time status updates are delivered to all stakeholders via Socket.IO
- Admin dashboard accurately reflects system-wide statistics and booking records
- Feedback collection and support ticket resolution are fully operational

The system was tested with multiple concurrent users across all three portals, and all critical workflows operated without errors.

4.2 Screenshots of the Application

The following screenshots demonstrate the key interfaces of the RailMitra application:

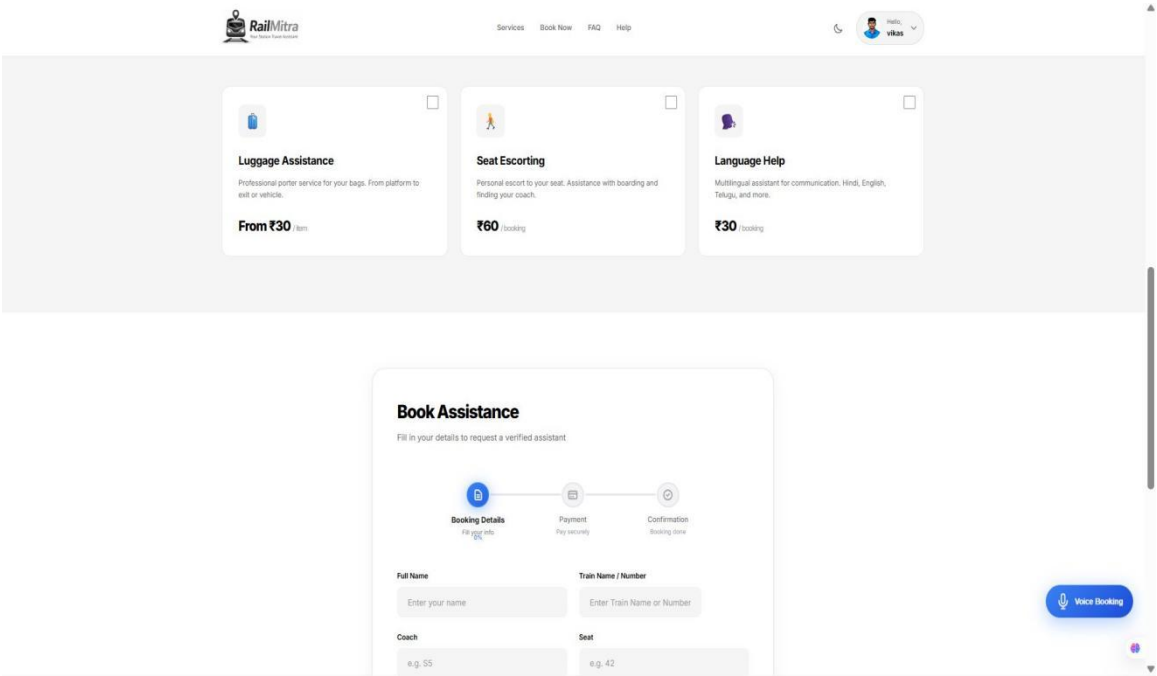

Figure 1: Passenger Portal – Service Selection and Booking Form

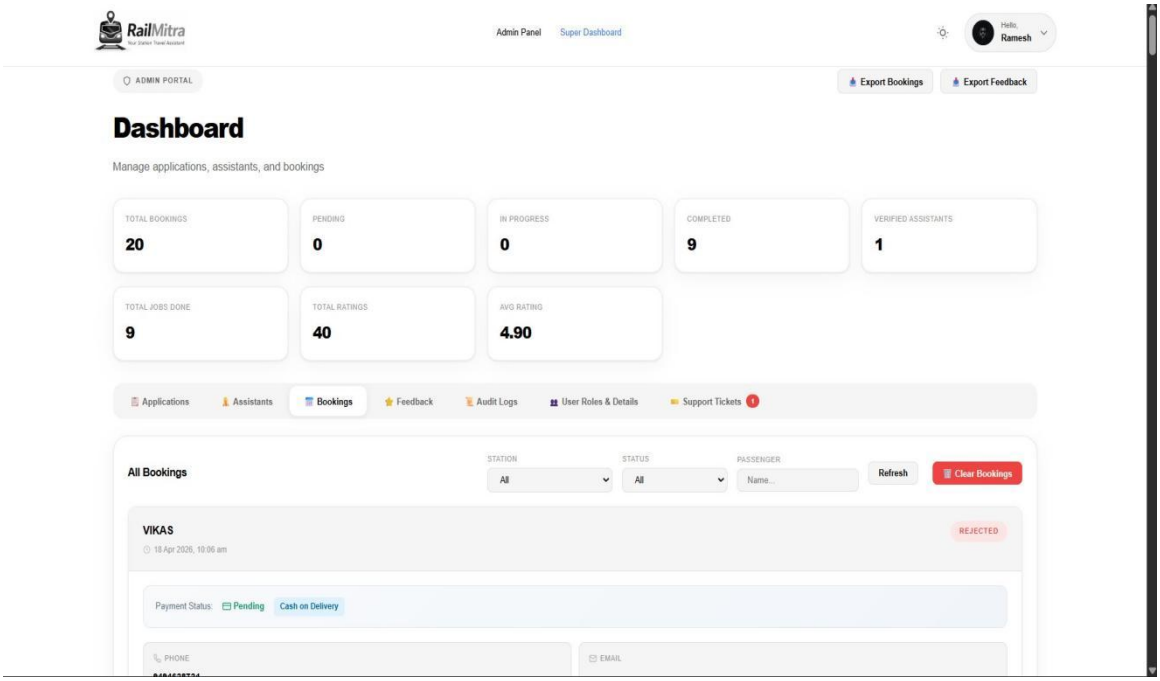

Figure 2: Admin Portal – Dashboard with Booking Management and Statistics

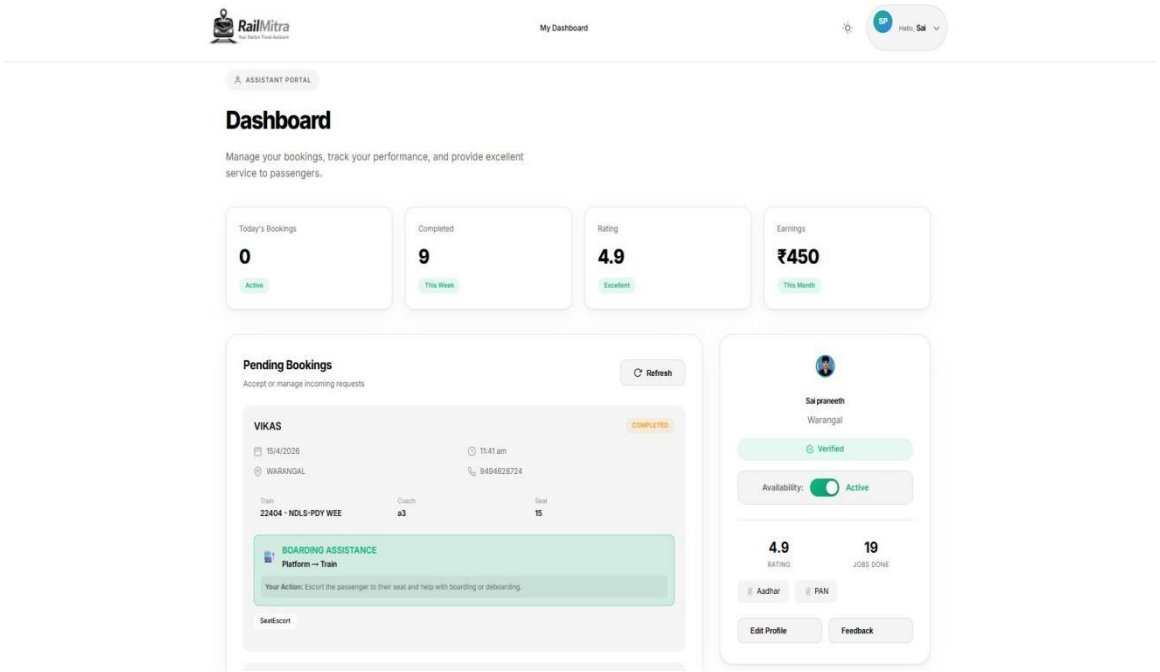

Figure 3: Assistant Portal – Task Management and Performance Dashboard

4.3 Workflow Example

A typical RailMitra service workflow proceeds as follows:

- Passenger selects a service (e.g., Luggage Assistance, Seat Escorting, Language Help) from the service catalog
- Passenger fills in booking details including train number, coach, seat, and preferred time
- Assistant receives the booking notification and accepts the task
- Real-time updates are shared with the passenger via Socket.IO as the task progresses

- Service is completed and marked accordingly in the system
- Passenger submits feedback and rating, which is reflected in the assistant's profile

4.4 Performance Discussion

Node.js handles concurrent requests efficiently using its event-driven, non-blocking I/O model. This ensures that multiple passengers and assistants can interact with the system simultaneously without performance degradation.

MongoDB provides the schema flexibility required for heterogeneous booking data, allowing seamless addition of new service types and fields without schema migration overhead.

Socket.IO enables near-instantaneous real-time updates, with observed latency well within acceptable thresholds for the use case.

4.5 Limitations

- The system requires active internet connectivity for all features including real-time updates
- The platform has not been tested at very large scale (thousands of concurrent bookings)
- Current analytics capabilities are limited; advanced reporting features are planned for future versions
- Mobile application support is not yet available in the current iteration

CHAPTER 5: CONCLUSION AND FUTURE WORK

5.1 Summary

RailMitra successfully integrates booking management, assistant task handling, real-time communication, and administrative monitoring into a unified full-stack platform. The system was designed to address the genuine needs of railway passengers who require station assistance services, with particular focus on accessibility and ease of use.

The project was developed by a team of four students as part of the DevOps and Full Stack course at SR University, Warangal, under the guidance of Dr. N. Venkatesh.

5.2 Conclusion

The RailMitra project successfully demonstrates that integrating full-stack web technologies with real-time communication frameworks can significantly improve railway assistance service delivery. The role-based architecture ensures clear accountability at every stage of the service workflow, while the admin dashboard provides the operational visibility necessary for effective management.

The project validates the practical applicability of Node.js, Express.js, MongoDB, and Socket.IO as a cohesive technology stack for building scalable, real-time service management applications.

5.3 Future Scope

- AI-based assistant allocation using machine learning models to optimize task assignment based on proximity, expertise, and availability
- Native mobile application for Android and iOS to improve accessibility for passengers on the go
- Multilingual support to serve passengers across different linguistic backgrounds
- WhatsApp and SMS integration for booking confirmations and real-time notifications
- Advanced analytics dashboard with visual reporting for station management and performance insights
- Integration with Indian Railways APIs for live train status and PNR-based booking validation

REFERENCES

- Node.js Documentation – https://nodejs.org/en/docs
- MongoDB Official Documentation – https://www.mongodb.com/docs
- Express.js Documentation – https://expressjs.com
- Socket.IO Documentation – https://socket.io/docs
- Humble, J. & Farley, D. (2010). Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley.
- Pressman, R. S. (2014). Software Engineering: A Practitioner's Approach. McGraw-Hill Education.